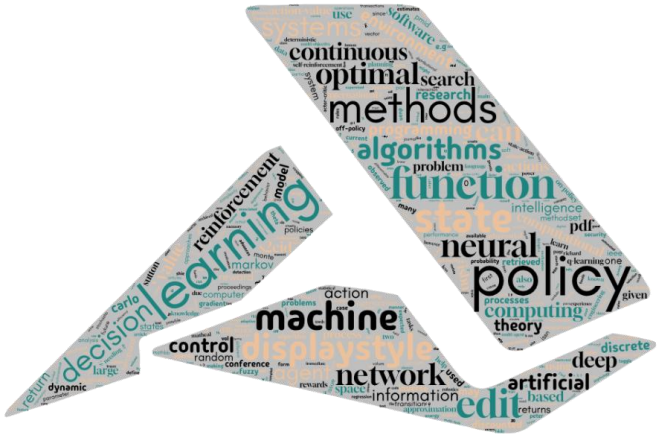
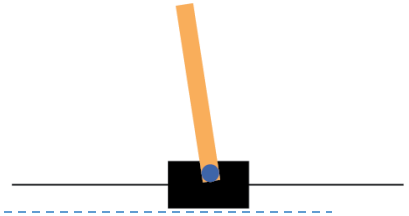


Reinforcement Learning Algorithms and Applications

Chapter 8. Soft Update of Deep Q Learning for CartPole

CartPole Environment Overview



- **CartPole-v1** is a classic **control problem** where an agent balances a pole on a moving cart.
- **State space** consists of **4 continuous variables**: cart position, cart velocity, pole angle, and pole angular velocity.
- **Action space** contains **2 discrete actions**: move cart left (0) or move cart right (1).
- **Episode termination** occurs when pole angle exceeds ±**12 degrees** or cart moves beyond ±**2.4 units**.
- **Maximum episode length** is **500 steps** with +1 reward per step survived.
- **Success criterion** typically requires maintaining balance for **200+ consecutive steps**.
- **Environment complexity** lies in the **unstable equilibrium** nature of the inverted pendulum.
- **Real-time rendering** available for visualization during training and demonstration phases.

Network Architecture

- **DQN Model** uses a **three-layer fully connected** neural network with **ReLU activations**.
- **Input layer** processes **n_observations** (4 for CartPole) state features.
- **Hidden layers** contain **128 neurons each** for feature extraction and representation learning.
- **Output layer** produces **Q-values** for all possible actions (2 for CartPole).
- **Forward pass** applies sequential transformations: *Linear* → *ReLU* → *Linear* → *ReLU* → *Linear*.
- **Network parameters** are optimized using **AdamW optimizer** with learning rate **3e-4**.
- **Architecture design** balances model capacity with computational efficiency.

Experience Replay Memory

- **ReplayMemory class** implements a **circular buffer** with maximum capacity of **10,000 transitions**.
- **Transition tuple** stores *(state, action, next_state, reward)* for each experience.
- **Memory sampling** randomly selects **batch_size** transitions to break correlation between consecutive experiences.
- **Deque structure** automatically removes oldest experiences when capacity is exceeded.
- **Random sampling** improves training stability by reducing temporal correlations.
- **Buffer capacity** should be large enough to store diverse experiences across different episodes.
- **Experience replay** enables the agent to learn from past experiences multiple times.

Action Selection Strategy

- **Epsilon-greedy policy** balances **exploration** and **exploitation** during training.
- **Epsilon decay formula**:
  $\epsilon = \epsilon_{end} + (\epsilon_{start} - \epsilon_{end}) \times e^{-\frac{steps}{decay}}$
- **Initial epsilon** ($\epsilon_{start} = 0.9$) encourages high exploration early in training.
- **Final epsilon** ($\epsilon_{end} = 0.01$) maintains minimal exploration after convergence.
- **Decay rate** ($decay = 2500$) controls the speed of transition from exploration to exploitation.
- **Training mode** uses epsilon-greedy with noise, while **demo mode** uses pure greedy policy.
- **Action selection** returns the action with highest Q-value when not exploring.
- **Random actions** are selected from the environment's action space during exploration.

Reward Rules

- **If the pole remains upright** → reward = +1
- **For every time step (step)** → +1 reward is given
- **When the episode ends** → no more rewards
- Episode Termination Conditions
  - The episode ends if:
    - The pole falls beyond a certain angle
    - The cart moves out of the allowed range
    - The maximum number of steps is reached (500 in CartPole-v1)

Q-Learning Update Process

- **Current Q-values** are computed using policy network: $Q(s, a; \theta)$
- **Target Q-values** follow **Bellman equation**: $r + \gamma \max_{a'} Q(s', a'; \theta_{target})$
- **Discount factor** $\gamma = 0.99$ weights future rewards in the learning process.
- **Batch processing** enables efficient parallel computation of Q-value updates.
- **Non-final states** are identified to handle terminal conditions properly.
- **Target network** provides stable Q-value targets by using older parameters.
- **Loss function** uses **SmoothL1Loss (Huber Loss)** for robust training against outliers.
  - **Small errors** → use L2 (squared error)
    Large errors → use L1 (absolute error)
  - Let the error be: $x = y_{pred} - y_{true}$
  - $\text{SmoothL1}(x) = \begin{cases} \frac{1}{2}x^2 & \textit{if } |x| < 1 \\ |x| - \frac{1}{2} & \textit{otherwise} \end{cases}$
- **Gradient clipping** prevents exploding gradients by limiting parameter updates to 100.

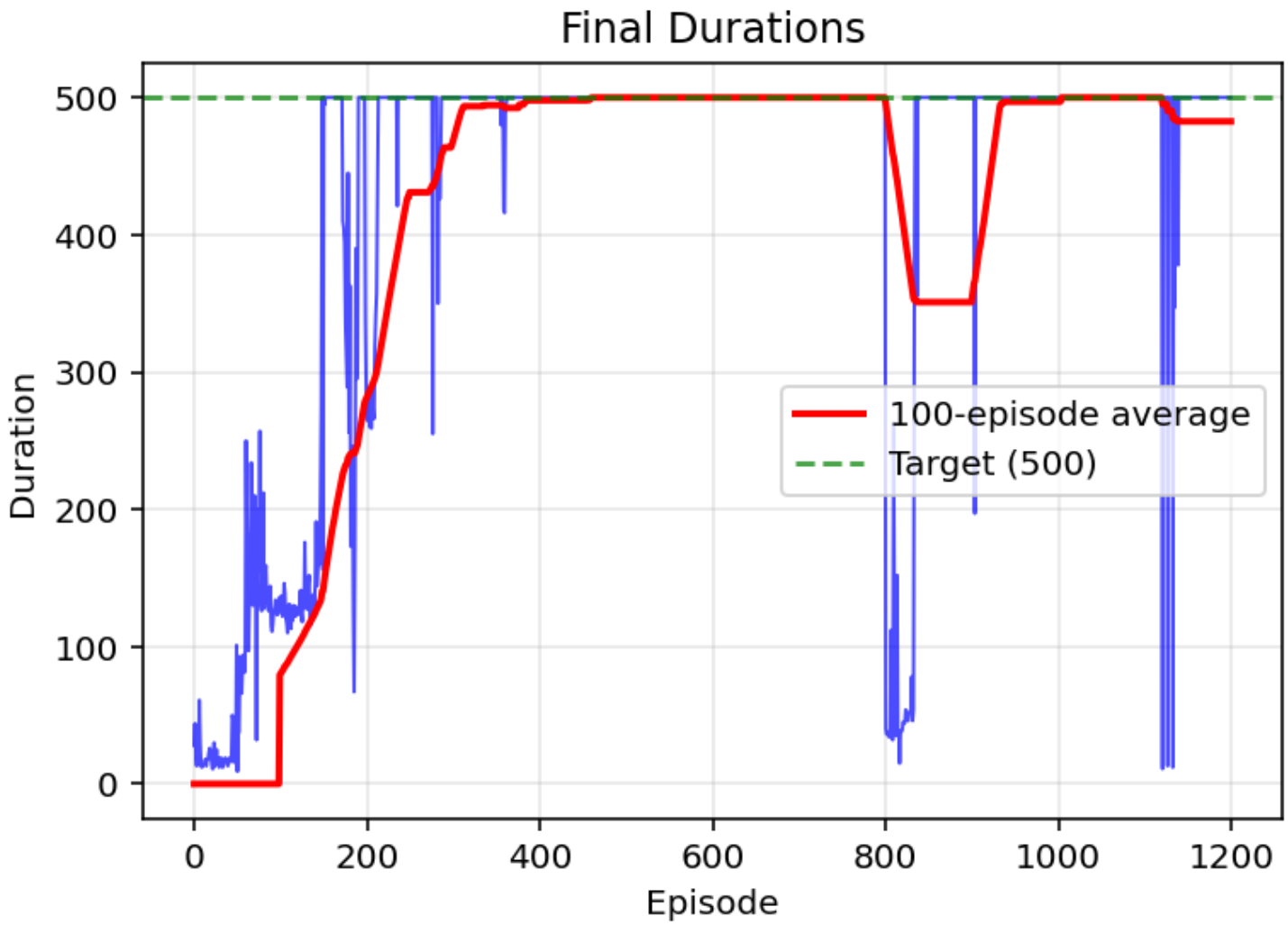
Soft Update Mechanism

- **Soft update** gradually transfers knowledge from policy network to target network.
- **Update formula**: $\theta_{target} = \tau \times \theta + (1 - \tau) \times \theta_{target}$
- **Tau parameter** ($\tau = 0.005$) controls the **update rate** - smaller values provide more stability.
- **Continuous updates** happen after every optimization step, not periodically.
- **Stability improvement**: Target network changes slowly, reducing training oscillations.
- **Parameter interpolation** ensures smooth transitions between network weights.
- **Soft updates** outperform hard updates by maintaining consistent target values.
- **Implementation** updates each layer's parameters independently using the same tau value.

Training Configuration

- **Batch size** of **128** provides good balance between computational efficiency and gradient stability.
- **Learning rate** of **3e-4** with **AdamW optimizer** and **AMSGrad** for stable convergence.
- **Replay memory capacity** of **10,000** transitions stores sufficient experience diversity.
- **Episode target** varies: **600 episodes** for GPU/MPS, **50 episodes** for CPU training.
- **Render settings** allow visualization every **10 episodes** with **0.02 second** delays.
- **Success threshold** defined as **200+ steps** for CartPole task completion.
- **Progress tracking** monitors average duration over last **50 and 100 episodes**.
- **Model checkpointing** automatically saves policy network, target network, and training statistics.

Training Result



Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.